Паттерны проектирования

Проектирование микросервисной архитектуры с использованием нотаций для системы «Электронная библиотека»

Домашнее задание №4

ВЫПОЛНИЛ: ШУМКОВ СЕРГЕЙ
ДАТА: 11 ИЮЛЯ 2026 Г.

Содержание

- Применимость паттернов проектирования
- Проектирование решений с паттернами
- Контрольные вопросы для самопроверки

Исходные данные

- Приобретён готовый каталог книг в формате EPUB (файлы + метаданные).
- Есть возможность докупать книги, поступающие ZIP-архивами (по несколько книг в архиве).
- **Технические ограничения:**
- Файлы хранятся в собственном S3-хранилище, метаданные - в БД.
- Для скачивания книг создаются временные ссылки.
- **Ключевые бизнес-процессы**
- Пополнение каталога контент-менеджером.
- Поиск и скачивание книг читателем.
- После прочтения читатель может оставить отзыв и поставить оценку. Отзыв проходит премодерацию.
- Читатель создаёт обращение (тикет), сотрудник техподдержки отвечает на обращение, созданное читателем.

Применимость паттернов проектирования

Паттерн	Применим/ не применим	Где может применяться	Какую проблему он решает	Как может изменить архитектуру
Одиночка	Применим	В микросервисах для создания пула соединений с базой данных	Предотвращает бесконтрольное открытие сотен сетевых сокетов к базе данных	Создает единую точку доступа к пулу соединений БД внутри одного инстанса сервиса.
Фабричный метод	Применим	В сервисе уведомлений для создания объектов отправки сообщений в зависимости от типа (Email, Push, SMS)	Позволяет легко расширять систему: если придется добавить отправку через Telegram	Скрывает детали создания объектов от остального кода микросервиса
Абстрактная фабрика	Применим	В сервисе хранения для создания семейства взаимосвязанных объектов для работы с разными облачными провайдерами	Код сервиса работает с абстрактными интерфейсами, не зная, куда именно физически отправляются файлы	Отвязывает логику микросервиса от работы с конкретным облачным вендором
Адаптер	Применим	В Notification Service для интеграции с разными SMS-провайдерами	Приводит несовместимые интерфейсы внешних систем к интерфейсу нашей системы	Создает защитный слой, который при изменений во внешних API позволяет менять код только адаптера
Декоратор	Применим	Логирование сервисов, сбор метрик	Позволяет динамически добавлять объектам новую обязанность без изменения их кода.	Очищает бизнес-код сервисов от инфраструктурных задач
Мост	Применим	В сервисе каталога для разделения способов их физического хранения	Без Моста пришлось бы создавать классы BookS3, BookLocal	Разделяет бизнес-сущности и системную инфраструктуру

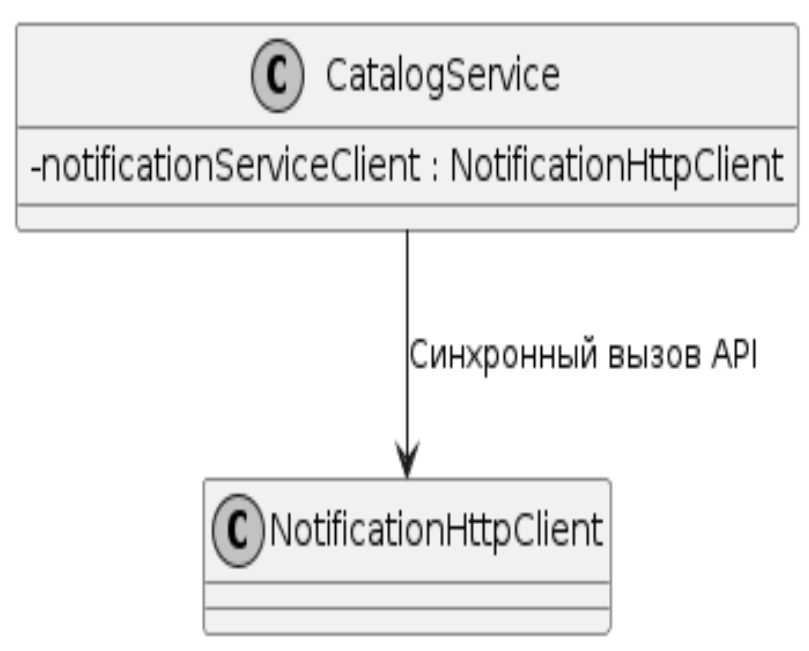
Применимость паттернов проектирования

Паттерн	Применим/ не применим	Где может применяться	Какую проблему он решает	Как может изменить архитектуру
Заместитель	Применим	API Gateway (внешний прокси для всей системы)	Решает задачи авторизации, кэширования ответов	Код целевого сервиса вообще не знает о проверках
Цепочка обязанностей	Применим	В контуре авторизации (AuthNZ)	Запрос проходит сквозь «звенья»: Валидность токена -> Проверка блокировки -> Проверка прав скачивать	Организует гибкий и настраиваемый пайплайн обработки запросов
Команда	Применим	В фоновом воркере, скачивающем книгу	Превращает действие (скачать книгу) в самостоятельный объект, что позволяет складывать запросы в очередь	Разрывает прямую связь между тем, кто инициировал действие (пользователь), и тем, кто его выполняет (фоновый воркер)
Стратегия	Применим	В логике расчета стоимости на книги в Catalog Service	Если у пользователя подписка «Премиум» — применяется одна стратегия расчета цены, если «Базовая» — другая	Новые правила расчетов добавляются в виде изолированных классов-стратегий
Наблюдатель	Применим	Взаимодействие между Catalog Service (Издатель) и сервисом Notification Service	Catalog Service не вызывает сторонние API напрямую, а лишь отправляет событие	Позволяет добавлять новые потребители данных без изменения кода источника.
Посетитель	Не применим	Паттерн требует, чтобы структура классов была стабильной	Изменение одного поля в карточке книги ломает код всех «посетителей»	На уровне микросервисов скорее принесет излишние усложнения

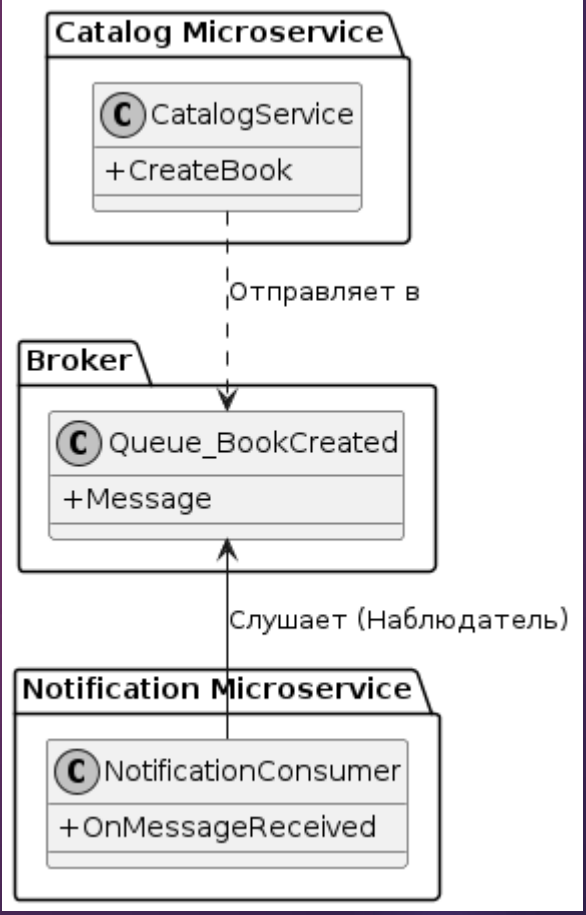
Проектирование решений с паттернами

- Сценарий 1: Оповещение смежных систем при добавлении книги в каталог

как было (без паттерна)



как стало (с паттерном)



При появлении новой книги Catalog Service больше никого не вызывает напрямую. Он генерирует событие и отправляет его в брокер.

Наблюдатель переносит оповещение на уровень инфраструктуры (через брокер), полностью зависимость между сервисами.

Альтернативы:
Цепочка обязанностей: Не подходит, так как сервисы должны работать параллельно и изолированно.

Компромиссы:
Минусы: Сообщение о новой книге у пользователя появится не мгновенно, а через промежуток времени после отправки события.

Плюсы: Отказоустойчивость

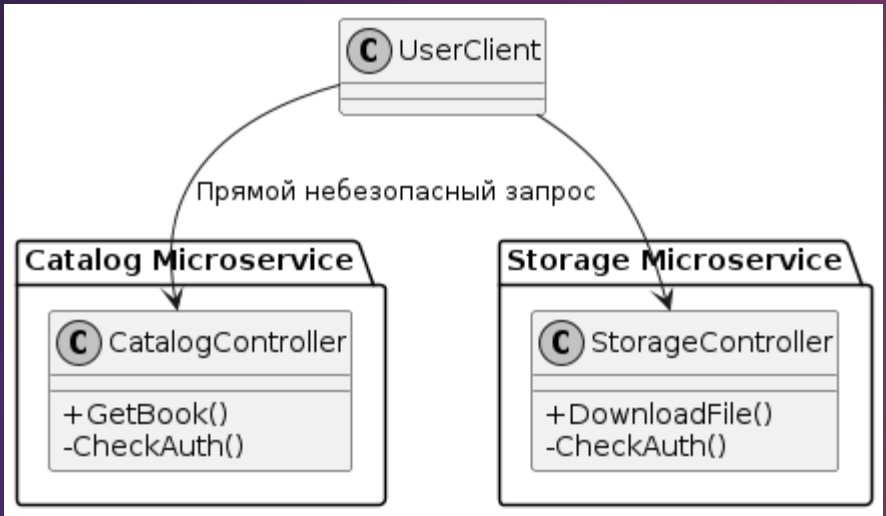
Связанность. Стала минимальной. Catalog Service физически ничего не знает о существовании других сервисов.

Расширяемость. Хорошая. Если завтра появится сервис рекомендаций, которому тоже нужны данные о новых книгах, мы просто подпишем его на очередь.

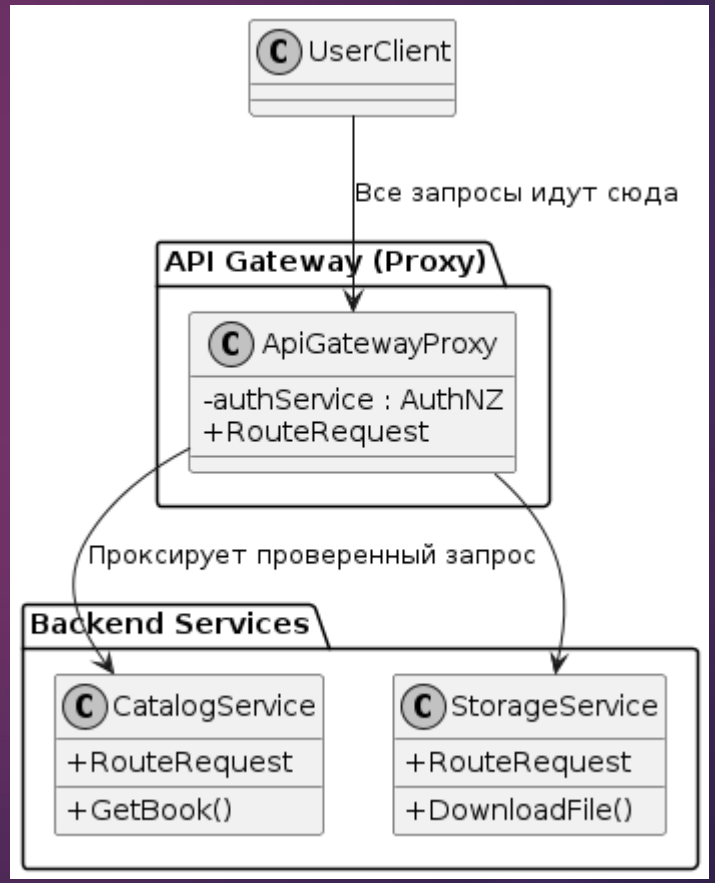
Тестируемость. Улучшилась. Catalog Service можно протестировать, просто проверив факт отправки сообщения в брокер.

- Сценарий 2: Перехват запросов для авторизации

как было (без паттерна)



как стало (с паттерном)



Клиент делает запросы не напрямую к сервисам, а на единую точку входа. API Gateway перехватывает трафик и централизованно проверяет токены.

Альтернативы:

Декоратор. Можно было обернуть декораторами внутри каждого сервиса. Но это не решает проблему на уровне сети: DDoS все равно долетал бы по сети до сервисов и грузил их ресурсы.

Компромиссы:

Минусы: Появляется лишний сетевой прыжок, что добавляет около 1–2 мс к задержке. При падении Gateway недоступна становится вся библиотека.

Плюсы: Бизнес-сервисы полностью изолированы от внешнего мира.

Связанность. Стала минимальной. Микросервисы больше не знают об алгоритмах валидации токенов.

Расширяемость. Хорошая. Можно добавить новые правила безопасности на шлюзе, не меняя код бизнес-сервисов.

Тестируемость. Улучшилась. Код бизнес-логики теперь тестируется в изоляции, в юнит-тестах не надо создавать фейковые кейсы безопасности.

Контрольные вопросы для самопроверки

- 1. Почему паттерны — это не готовые решения, а шаблоны?
  - Паттерны - это типичные способы решения часто встречающихся проблем при проектировании программ. Паттерн нельзя просто скопировать в проект в виде готового решения.
- 2. Когда не нужно применять паттерны?
  - Паттерны не нужно применять когда логика простая (CRUD-операции). Применение паттернов в таком случае приведет к усложненности кода, большому количеству классов, длительности разработки.
- 3. Чем отличается Стратегия от Фабричного метода?
  - Фабричный метод – создает объекты (порождающий паттерн), а Стратегия – изменяет поведение уже созданного объекта (поведенческий паттерн). Например, с помощью фабричного метода мы создаем новые способы отправки сообщений. А с помощью стратегии производим расчет стоимости книги для разных видов подписок.

- 4. Почему Адаптер важен для интеграций?
  - Система постоянно общается со сторонними сервисами, интерфейсы которых мы не контролируем. Адаптер приводит чужие форматы данных к формату нашей системы. При изменении формата данных внешней системы придется изменить только код адаптера.
- 5. Как Наблюдатель связан с событийно архитектурой?
  - Событийная архитектура — это подход, при котором компоненты системы взаимодействуют между собой через публикацию и обработку событий (сигналов об изменении состояния). Микросервисы-подписчики выступают Наблюдателями, они слушают брокер и реагируют на события изолированно друг от друга.
- 6. Где применим Заместитель в микросервисах?
  - Классическим Заместителем является API Gateway. Он перехватывает все запросы к системе, выполняет авторизацию через AuthNZ, логирование, ограничение частоты запросов. После этого перенаправляет запросы к сервисам,

- 7. В чем риск паттерна Одиночка в распределенной системе?
  - Одиночка гарантирует уникальность экземпляра объекта строго внутри одного процесса. В микросервисах один и тот же сервис запускается во множестве экземпляров (подов в Kubernetes. В масштабе всей системы будет столько «одиночек», сколько запущено реплик сервиса.
- 8. В чем отличие паттернов Адаптер от Моста?
  - **Адаптер** применяется к *уже существующему* коду, чтобы заставить работать две готовые, но несовместимые системы.
  - **Мост** закладывается *на старте* собственной системы, чтобы разделить абстракцию (бизнес-логику) и реализацию (инфраструктуру) позволяя изменять и развивать их абсолютно независимо друг от друга.